Restaurant Order & Inventory Management System

SHAIK NASEEBA

104R BATCH

10000 CODERS

RESTAURANT ORDER & INVENTORY MANAGEMENT SYSTEM

A DATABASE MANAGEMENT SYSTEM PROJECT

PROJECT REPORT

SUBMITTED BY

SHAIK NASEEBA

PROJECT TITLE

Restaurant Order & Inventory Management System

TECHNOLOGY USED

MySQL Database Management System

TRAINING INSTITUTE

10 CODERS COACHING CENTER

ACADEMIC YEAR

2026

DATABASE MANAGEMENT SYSTEM PROJECT REPORT

CERTIFICATE

This is to certify that **SHAIK NASEEBA** has successfully completed

the project entitled:

"RESTAURANT ORDER & INVENTORY MANAGEMENT SYSTEM"

at **10K CODERS COACHING CENTER** during the academic year **2026.**

This project has been completed as part of **Database Management System** training and project work.

The project focuses on designing and managing a relational database for restaurant ordering and inventory operations, including customers, menu items, orders, order items, inventory, suppliers, and purchases.

Introduction

Introduction of SQL :

SQL (Structured Query Language) is a standard programming language used to communicate with relational databases. It is mainly used to create, store,

retrieve, update, and manage data in a database.

SQL provides a simple and efficient way to work with large amounts of structured data. It allows users to create databases and tables, insert records, modify existing information, delete unwanted records, and retrieve specific data according to requirements.

SQL also supports relationships between tables and provides different

operations for combining and analyzing data. Important SQL concepts include data definition, data manipulation, data retrieval, joins, aggregate functions, subqueries, views, and stored procedures.

SQL is widely used with relational database management systems such as

MySQL, Microsoft SQL Server, Oracle Database, and PostgreSQL.

The major advantages of SQL are its simplicity, flexibility, efficiency, and ability

to manage large amounts of data. It is an important technology for database administrators, software developers, data analysts, and other professionals who work with databases**.**

INTRODUCTION TO RESTAURANT ORDER & INVENTORY MANAGEMENT DATABASE

A Restaurant Order & Inventory Management Database is a structured system used to store, organize, and manage information generated during restaurant operations. It helps maintain customer information, menu details, orders, individual order items, ingredient stock, supplier information, and purchase transactions in an organized manner.

The main purpose of the database is to connect the customer ordering process with the ingredient inventory and purchasing process. It allows restaurant information to be maintained consistently and makes it easier to retrieve and analyze operational data.

The database structure used in this project contains seven major components: Customers, Menu Items, Orders, Order Items, Inventory, Suppliers, and Purchases. These components work together to represent the main activities of a restaurant.

A well-designed restaurant database makes it easier to analyze sales and inventory activity. It can be used to identify top-selling dishes, monthly revenue, frequent customers, frequently restocked ingredients, supplier quantities, and the menu category generating the highest revenue.

DATABASE DESIGN

The database consists of seven major tables: Customers, Menu_Items, Orders, Order_Items, Inventory, Suppliers, and Purchases. The relationships between these tables allow the system to represent customer ordering, menu sales, ingredient stock, supplier information, and purchasing activities.

CUSTOMERS TABLE

The Customers table stores information about restaurant customers.

Main information:

- Customer ID
- Name
- Phone
- Email
- Address

Purpose: The Customer ID uniquely identifies each customer. This table acts as the central customer reference for order records.

MENU_ITEMS TABLE

The Menu_Items table stores information about dishes served by the restaurant.

Main information:

- Item ID
- Item Name
- Category
- Price
- Availability Status

Purpose: The Item ID uniquely identifies each dish. Category groups dishes, price stores the selling price, and availability status indicates whether the dish is available or out of stock.

ORDERS TABLE

The Orders table stores order details for customers.

Main information:

- Order ID
- Customer ID
- Order Date
- Total Amount
- Status

Purpose: The Order ID uniquely identifies each order. Customer ID connects an order with the customer who placed it. Status identifies whether the order is placed, completed, or cancelled.

ORDER_ITEMS TABLE

The Order_Items table links orders with menu items.

Main information:

- Order Item ID
- Order ID
- Item ID
- Quantity
- Subtotal

Purpose: This table allows one order to contain multiple dishes and allows a menu item to appear in multiple orders. Quantity records how many units were ordered and subtotal represents the amount for that item.

INVENTORY TABLE

The Inventory table tracks stock of ingredients used in dishes.

Main information:

- Item Code
- Ingredient Name
- Quantity in Stock

- Unit
- Reorder Level

Purpose: The item code identifies an ingredient record. Quantity in stock represents the current quantity, unit identifies kg, liters, pieces, or another measurement, and reorder level indicates when additional stock is required.

SUPPLIERS TABLE

The Suppliers table stores supplier information for restocking ingredients.

Main information:

- Supplier ID
- Supplier Name
- Phone
- Email

Purpose: Supplier ID uniquely identifies a supplier and is used to connect suppliers with purchase records.

PURCHASES TABLE

The Purchases table tracks ingredient purchases from suppliers.

Main information:

- Purchase ID
- Supplier ID
- Item Code
- Purchase Date
- Quantity
- Total Cost

Purpose: Purchase ID uniquely identifies a purchase. Supplier ID connects the purchase to a supplier, while Item Code identifies the ingredient purchased.

RELATIONSHIPS BETWEEN TABLES

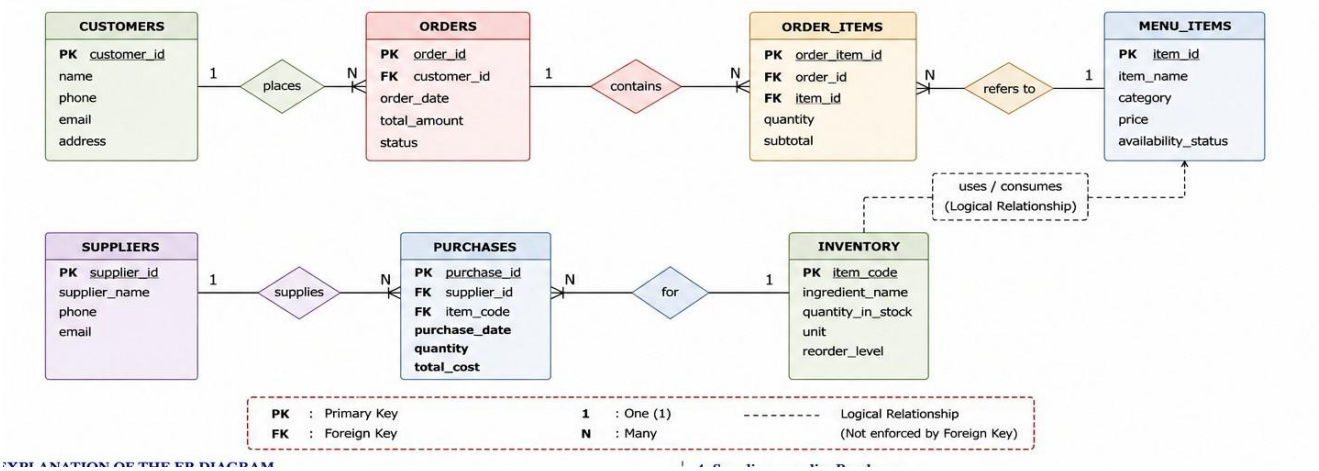
The database follows a relational structure. Primary keys uniquely identify records, while foreign keys connect related records across tables.

- Customers → Orders: One customer can place multiple orders.
- Orders → Order_Items: One order can contain multiple order items.
- Menu_Items → Order_Items: One menu item can appear in multiple order items.
- Suppliers → Purchases: One supplier can have multiple purchase transactions.
- Inventory → Purchases: One ingredient can appear in multiple purchase transactions.
- Orders and Menu_Items are connected through Order_Items, creating a many-to-many relationship.

These relationships help maintain data consistency and allow information from multiple tables to be analyzed together.

ER DIAGRAM

The Entity Relationship (ER) diagram for the Restaurant Order & Inventory Management System is shown below.



EXPLANATION OF THE ER DIAGRAM

CUSTOMERS AND ORDERS

The Customers table stores the details of the people who place orders in the restaurant. The Orders table stores the orders made by those customers. The

customer_id in Orders connects each order to the customer who placed it. One customer can place many orders, but each order belongs to one customer.
This is a one-to-many relationship.

ORDERS AND ORDER_ITEMS

The Orders table contains the main information about an order, while Order_Items contains the individual items included in that order. The order_id is used to connect the two tables. One order can contain many order items, while each order item belongs to one order.

ORDER_ITEMS AND MENU_ITEMS

The Menu_Items table contains the dishes available in the restaurant. Order_Items records the dishes selected by the customer. The item_id connects these two tables. The same menu item can be included in many different orders, while each order item refers to one menu item.

SUPPLIERS AND PURCHASES

The Suppliers table stores information about the suppliers who provide ingredients to the restaurant. The Purchases table records the stock purchased from those suppliers. The supplier_id connects a purchase with its supplier. One supplier can have many purchase records.

INVENTORY AND PURCHASES

The Inventory table keeps information about ingredients and the available stock. When the restaurant buys ingredients, the purchase is recorded in Purchases. The item_code connects the purchased item with the corresponding inventory item. This helps the restaurant keep track of stock received from purchases.

INVENTORY AND MENU_ITEMS

Inventory and Menu_Items are related from a business point of view because ingredients in inventory are used to prepare menu items. The given database structure does not contain a separate recipe or ingredient-mapping table, so this connection is treated as a logical relationship rather than a direct foreign-key relationship.

PRIMARY KEYS AND FOREIGN KEYS

The primary key uniquely identifies each record in a table. Foreign keys are used to connect related tables. For example, customer_id connects Customers with Orders, order_id connects Orders with Order_Items, item_id connects Order_Items with Menu_Items, and supplier_id connects Suppliers with Purchases.

OVERALL ER DIAGRAM

The ER diagram gives a clear picture of how the restaurant database works as a single system. Customers place orders, orders contain order items, and order items refer to menu items. On the inventory side, suppliers provide ingredients through purchases, and those ingredients are maintained in the Inventory table. These relationships help the restaurant manage orders and stock in an organized way.

BASIC DATABASE OPERATIONS

The project includes basic operations for working with restaurant information. Sample data is maintained across all seven tables, and the system retrieves records according to business requirements.

INSERT OPERATION

The INSERT operation is used to add new records to a table. In this project it is used to populate Customers, Menu_Items, Orders, Order_Items, Inventory, Suppliers, and Purchases with sample information. It is important because all later analysis depends on having meaningful records in the database.

SELECT OPERATION

The SELECT operation is used to retrieve information from database tables. It can display all columns or only selected columns. In this project it is used whenever restaurant information needs to be viewed or analyzed.

WHERE CLAUSE

The WHERE clause filters records according to a condition. It is useful when only certain records are required. For example, the project uses filtering to find menu items above a specified price, customers who placed orders in October 2025, menu items marked Out of Stock, and completed orders.

COMPARISON OPERATORS

Comparison operators such as greater than, less than, equal to, and their related forms compare column values with a required condition. They are useful for price filtering, stock-level checking, date filtering, and identifying orders above a particular amount.

DATE FILTERING

Date conditions are used to restrict records to a particular period. In this project, date filtering is required for finding customers who placed orders in October 2025 and for producing monthly reports.

JOIN OPERATIONS

Joins are used to combine information from multiple related tables. Restaurant information is distributed across several tables, so joins are necessary to create meaningful combined results.

INNER JOIN

An INNER JOIN returns matching records from the related tables. It is suitable when the project needs information that exists in both sides of a relationship, such as customers with their orders or orders with their ordered menu items.

LEFT JOIN

A LEFT JOIN keeps every record from the left table and includes matching information from the right table when available. This is especially useful when the report should also show records with no matching activity, such as menu items that have not been ordered or customers who have no orders.

JOIN USING PRIMARY KEY AND FOREIGN KEY

The relationship condition connects the primary key of one table with the corresponding foreign key of another table. For example, customer information is connected to orders through customer ID, while order items are connected to orders and menu items through their respective IDs.

The project uses joins to list customer names with ordered item names, display order details with item names and quantities, show orders with customer details, connect suppliers with ingredients, and display menu item categories used in orders.

AGGREGATE FUNCTIONS

Aggregate functions summarize multiple records and return a single calculated value or a grouped summary. They are important for restaurant reports because management usually needs totals, averages, maximum values, and counts rather than every individual record.

COUNT() FUNCTION

COUNT() counts records or values. In this project it can count the total number of orders placed in a month. It can also be used when counting the number of records belonging to a particular group.

SUM() FUNCTION

SUM() adds numeric values together. It is used to calculate total revenue, total quantities, total purchase quantities, or other numerical totals. For restaurant analysis, SUM is especially useful for understanding total sales and stock movement.

AVG() FUNCTION

AVG() calculates the average of numeric values. The project uses this concept to find the average order amount. Average values are also useful as comparison points for identifying orders above average.

MAX() FUNCTION

MAX() returns the largest value in a selected set of values. In this project it can identify the most expensive dish and can also support analysis where the highest quantity or cost is required.

GROUP BY CLAUSE

GROUP BY combines records having the same value into groups before aggregate functions are applied. It is useful for calculating totals for each ingredient, each month, each customer, each menu category, or each supplier.

ORDER BY CLAUSE

ORDER BY sorts the result. It can arrange values in ascending or descending order. It is important for reports such as top-selling dishes, highest-spending customers, highest-quantity suppliers, and categories with the highest revenue.

LIMIT CLAUSE

LIMIT restricts the number of records returned. For example, a Top 5 selling dishes report requires only the first five records after sorting by sales quantity.

SUBQUERIES

A subquery is a query placed inside another query. It is used when one result is needed to answer another question. The restaurant project uses subqueries for advanced analysis.

NOT IN

NOT IN checks whether a value does not belong to a set of values returned by another operation. It is useful for finding customers who have not placed any orders and menu items that were never ordered.

SUBQUERY FOR REORDER LEVEL

A subquery can compare current inventory information against the required reorder level. This helps identify ingredients whose stock has fallen below the level defined for restocking.

SUBQUERY FOR AVERAGE

A subquery can first calculate the average order amount and then use that result as a comparison value. This makes it possible to identify orders whose total amount is above the average.

SUBQUERY FOR RECENT ACTIVITY

A subquery can be used to identify suppliers that do not appear in recent purchase or delivery records. This helps management identify suppliers who have not recently supplied ingredients.

UPDATE AND DELETE OPERATIONS

UPDATE

UPDATE modifies existing records without removing them. In this project it is used to change menu item availability after an order is completed, update supplier contact details, and increase or adjust ingredient stock after a new purchase.

DELETE

DELETE removes records that are no longer required. The project includes deletion of cancelled orders and customers with no order history. Deletion should be performed carefully because removing related records can affect database relationships.

ADVANCED DATABASE FEATURES

TRIGGERS

A trigger is an automatic database action that executes when a specified event occurs on a table. In this project, a trigger is proposed to reduce the inventory quantity automatically when an order is placed. This connects sales activity with stock management without requiring a separate manual stock update.

VIEWS

A view is a virtual table based on information from one or more tables. Views provide a customized and reusable representation of data. The project requires a combined order report containing customer information, ordered items, and total amount. A view can make this frequently required report easier to access.

STORED PROCEDURES

A stored procedure is a named, reusable group of database operations stored inside the database. Procedures are useful when the same task must be performed repeatedly. The project requires a procedure to generate a daily sales summary, so management can obtain routine sales information in a consistent manner.

TRANSACTIONS

A transaction groups related database operations into one logical unit. The project requires the order and its order items to be added only when all required items are available. This protects consistency by preventing a partially completed order from being stored when the required conditions are not satisfied.

REPORTS AND ANALYSIS

The database can generate several useful reports. These reports use filtering, joins, aggregate functions, grouping, sorting, and other database methods to convert stored operational information into meaningful business information.

TOP 5 SELLING DISHES BY QUANTITY

This report identifies the five menu items with the highest total quantity ordered. The method is to connect order items with menu items, group the records by dish, add the quantities using SUM(), sort the totals from highest to lowest using ORDER BY, and restrict the output to five records using LIMIT.

MONTHLY REVENUE REPORT FOR 2025

This report summarizes revenue month by month during 2025. Order dates are used to identify the month, completed sales are considered for revenue analysis, amounts are grouped by month, and SUM() is used to calculate the revenue total for each month. Sorting by month presents the report in chronological order.

MOST FREQUENT CUSTOMERS AND TOTAL SPENDING

This report identifies customers who place orders frequently and shows their total spending. Customers are connected with orders, records are grouped by customer, COUNT() measures order frequency, and SUM() calculates total spending. ORDER BY can arrange customers from the highest activity or spending to the lowest.

INGREDIENTS MOST FREQUENTLY RESTOCKED

This report identifies ingredients that appear frequently in purchase transactions. Purchase records are grouped by ingredient and COUNT() can measure the number of restocking transactions. Sorting the results helps identify ingredients that require frequent purchasing.

SUPPLIER PROVIDING THE HIGHEST QUANTITY

This report identifies the supplier associated with the highest total supplied quantity. Purchase records are grouped by supplier and SUM() is used to calculate the total quantity supplied by each supplier. ORDER BY can arrange suppliers by quantity and LIMIT can identify the highest one.

CATEGORY GENERATING THE HIGHEST REVENUE

This report determines which menu category contributes the highest revenue. Order items are connected to menu items, items are grouped by category, and the relevant sales amounts are summarized using SUM(). The categories are then sorted from highest to lowest revenue to identify the leading category.

FINAL OUTPUTS:

SELECT * FROM Customers;

customer_id	name	phone	email	address
1	Aarav Kumar	9876500001	aarav@gmail.com	Hyderabad
2	Priya Sharma	9876500002	priya@gmail.com	Secunderabad
3	Rahul Reddy	9876500003	rahul@gmail.com	Hyderabad
4	Sneha Patel	9876500004	sneha@gmail.com	Warangal
5	Vikram Rao	9876500005	vikram@gmail.com	Hyderabad
NULL	NULL	NULL	NULL	NULL

SELECT * FROM Menu_Items;

item_id	item_name	category	price	availability_status
101	Paneer Tikka	Starter	280.00	Available
102	Chicken Biryani	Main Course	350.00	Available
103	Veg Fried Rice	Main Course	240.00	Available
104	Masala Dosa	Main Course	180.00	Available
105	Gulab Jamun	Dessert	120.00	Out of Stock
NULL	NULL	NULL	NULL	NULL

SELECT * FROM Orders;

order_id	customer_id	order_date	total_amount	status
1001	1	2025-10-02	630.00	Completed
1002	2	2025-10-05	350.00	Placed
1003	1	2025-10-11	460.00	Completed
1004	3	2025-10-18	700.00	Completed
1005	4	2025-10-25	120.00	Cancelled
NULL	NULL	NULL	NULL	NULL

SELECT * FROM Order_Items;

order_item_id	order_id	item_id	quantity	subtotal
5001	1001	101	1	280.00
5002	1001	102	1	350.00
5003	1002	102	1	350.00
5004	1003	103	1	240.00
5005	1003	104	1	180.00
5006	1004	102	2	700.00
NULL	NULL	NULL	NULL	NULL

SELECT * FROM Inventory;

item_code	ingredient_name	quantity_in_stock	unit	reorder_level
201	Rice	40	kg	10
202	Chicken	25	kg	8
203	Paneer	12	kg	5
204	Cooking Oil	18	liters	6
205	Sugar	4	kg	5
NULL	NULL	NULL	NULL	NULL

SELECT * FROM Suppliers;

supplier_id	supplier_name	phone	email
301	Fresh Foods Suppliers	9000000001	freshfoods@gmail.com
302	Hyderabad Agro	9000000002	agro@gmail.com
303	Daily Dairy Supply	9000000003	dairy@gmail.com
304	Kitchen Essentials	9000000004	kitchen@gmail.com
305	City Wholesale	9000000005	citywholesale@gmail.com
NULL	NULL	NULL	NULL

SELECT * FROM Purchases;

purchase_id	supplier_id	item_code	purchase_date	quantity	total_cost
4001	301	201	2025-10-01	50	3000.00
4002	302	202	2025-10-03	20	7000.00
4003	303	203	2025-10-04	10	2500.00
4004	304	204	2025-10-08	25	3500.00
4005	305	205	2025-09-20	10	600.00
NULL	NULL	NULL	NULL	NULL	NULL

FUNCTIONS AND METHODS USED IN THE PROJECT

The following functions, clauses, operators, and database methods are directly relevant to the tasks specified in the restaurant project document.

Function / Method	Purpose	Use in This Project
INSERT	Adds new records.	Creates sample restaurant data.
SELECT	Retrieves stored records.	Displays customers, menu items, orders, inventory, suppliers, and purchases.
WHERE	Filters records by a condition.	Price, date, availability, order status, and stock filtering.
Comparison operators	Compare values.	Find prices above a value, dates in a period, or stock below a level.
INNER JOIN	Combines matching rows.	Connects customers, orders, order items, menu items, suppliers, and inventory.
LEFT JOIN	Keeps all rows from the left table.	Useful for finding records with no matching orders or activity.
COUNT()	Counts records/values.	Order counts, customer frequency, restocking frequency.
SUM()	Adds numeric values.	Revenue, quantities, spending, and supplier totals.
AVG()	Calculates an average.	Average order amount and above-average comparisons.
MAX()	Returns the highest value.	Most expensive menu item and highest-value analysis.
GROUP BY	Creates groups for summary calculations.	Monthly, customer, ingredient, supplier, and category summaries.
ORDER BY	Sorts results.	Top-selling dishes,

		spending, quantities, and revenue rankings.
LIMIT	Restricts output rows.	Top 5 dishes and highest supplier/category results.
Subquery	Uses one result inside another operation.	Inactive customers, never-ordered dishes, reorder analysis, and above-average orders.
NOT IN	Excludes values returned by another set.	Customers without orders and menu items never ordered.
UPDATE	Changes existing records.	Availability, supplier contacts, and stock updates.
DELETE	Removes records.	Cancelled orders and customers with no order history.
TRIGGER	Automatically performs an action after a database event.	Automatically reduces inventory when an order is placed.
VIEW	Creates a reusable virtual representation.	Combined order report.
STORED PROCEDURE	Stores reusable database logic.	Daily sales summary.
TRANSACTION	Groups related operations into one logical unit.	Adds an order and order items only when all items are available.

FUTURE ENHANCEMENTS

- Online customer ordering and reservation management.
- Online payment integration.
- Real-time inventory monitoring.
- Automatic low-stock notifications.
- Supplier performance tracking.
- Employee and staff management.
- Delivery and order tracking.

- Customer feedback and rating management.
- Advanced sales dashboards and real-time analytics.
- Mobile application integration.
- Menu recommendation features.
- Integration with accounting and billing systems.

PROJECT OUTCOME

- A structured restaurant database is designed.
- Customer information can be managed efficiently.
- Menu items and availability can be maintained.
- Orders and individual order items can be tracked.
- Ingredient stock and reorder levels can be monitored.
- Supplier and purchase activities can be recorded.
- Information from multiple related tables can be combined and analyzed.
- Sales, customer, inventory, and supplier reports can be generated.
- Important SQL database concepts can be demonstrated through a practical real-world application.

This gives a complete conceptual and functional foundation for a Restaurant Order & Inventory Management Database System.

CONCLUSION

The Restaurant Order & Inventory Management Database provides a structured approach for storing and managing information generated during restaurant operations. The project covers seven important entities: Customers, Menu_Items, Orders, Order_Items, Inventory, Suppliers, and Purchases. These entities are connected through relationships that allow the database to represent customer ordering and ingredient restocking activities.

The system demonstrates important database concepts including basic operations, joins, aggregate functions, subqueries, updates, deletes, views, stored procedures, triggers, and transactions. It also supports analytical reports such as top-selling dishes, monthly revenue, frequent customers and their spending, frequently restocked ingredients, supplier quantity analysis, and category-wise revenue.

Overall, the project demonstrates how a relational database can be designed and used to manage restaurant information efficiently, monitor inventory, understand sales performance, maintain supplier records, and generate meaningful reports for better business decisions.

SHAIK NASEEBA

104R BATCH

10000 CODERS